

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CƠ BẢN I

THỰC TẬP CƠ SỞ



BÁO CÁO TUẦN

Đề tài:

Xây dựng website tuyển dụng việc làm

Giảng viên hướng dẫn	: Kim Ngọc Bách
Họ và tên sinh viên	: Đỗ Bình Minh
Ngày sinh	: 24/12/2005
Số điện thoại	: 0329279863
Mã sinh viên	: B23DCCN542
Lớp	: D23CQCN10-B
Nhóm lớp	: 11

Hà Nội – 2026

Mục lục

1. Mục tiêu công việc trong tuần.....	3
2. Các công việc đã thực hiện trong tuần	3
2.1. Tính năng duyệt và từ chối (Trang quản lý CV).....	3
2.1.1. Xử lý bên phía Frontend.....	3
2.1.2. Xử lý bên phía Backend	4
2.1.3. Kết quả	10
2.2. Tính năng xóa (Trang quản lý CV)	11
2.2.1. Xử lý bên phía Frontend.....	11
2.2.2. Xử lý bên phía Backend	12
2.2.3. Kết quả	14
2.3. Quản lý CV đã gửi.	15
2.3.1. Xử lý bên phía Frontend.....	15
2.3.2. Xử lý bên phía backend.....	18
2.3.3. Kết quả	23
3. Kết quả đạt được	23
4. Khó khăn, vướng mắc	24
5. Hướng giải quyết.....	24
6. Kế hoạch tuần tiếp theo	25

1. Mục tiêu công việc trong tuần

Trong tuần này, mục tiêu chính là tiếp tục hoàn thiện các chức năng quản lý CV trong website tuyển dụng việc làm. Sau khi đã xây dựng chức năng xem chi tiết CV ở tuần trước, tuần này em tập trung phát triển thêm các thao tác xử lý CV cho cả phía nhà tuyển dụng và phía ứng viên.

Cụ thể, công việc cần thực hiện bao gồm:

- Xây dựng chức năng duyệt và từ chối CV tại trang quản lý CV của công ty.
- Cho phép nhà tuyển dụng cập nhật trạng thái CV sau khi xem xét hồ sơ ứng viên.
- Xây dựng chức năng xóa CV tại trang quản lý CV của công ty.
- Kiểm tra quyền thao tác, đảm bảo công ty chỉ được cập nhật hoặc xóa CV ứng tuyển vào công việc thuộc công ty mình.
- Xây dựng trang quản lý CV đã gửi dành cho ứng viên.
- Hiển thị danh sách các CV mà ứng viên đã ứng tuyển, bao gồm thông tin công việc, công ty, mức lương, hình thức làm việc và trạng thái xử lý CV.

2. Các công việc đã thực hiện trong tuần

2.1. Tính năng duyệt và từ chối (Trang quản lý CV)

2.1.1. Xử lý bên phía Frontend

```
export const CVList = () => {
  const [cvList, setCVList] = useState<any[]>([]);
  const [countUpdate, setCountUpdate] = useState(0);

  useEffect(() => {
    fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/cv/list`, {
      method: "GET",
      credentials: "include", // Gửi kèm cookie
    })
      .then(res => res.json())
      .then(data => {
        if (data.code === "success") {
          setCVList(data.cvs);
        }
      })
  }, [countUpdate]);

  const handleChangeStatus = (id: string, status: string) => {
    const dataFinal = {
      status: status
    };

    fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/cv/change-status/${id}`, {
      method: "PATCH",
      credentials: "include", // Gửi kèm cookie
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(dataFinal)
    })
      .then(res => res.json())
      .then(data => {
        if (data.code === "error") {
          toast.error(data.message);
        }

        if (data.code === "success") {
          toast.success(data.message);
          setCountUpdate(countUpdate + 1);
        }
      })
  }
}
```

Code này là component React/Next.js để **hiển thị danh sách CV và đổi trạng thái CV**.

CVList là component chính.

cvList lưu danh sách CV lấy từ API:

```
const [cvList, setCVList] = useState<any[]>([]);
```

countUpdate dùng để ép useEffect chạy lại sau khi cập nhật trạng thái:

```
const [countUpdate, setCountUpdate] = useState(0);
```

useEffect sẽ gọi API:

```
/company/cv/list
```

Nếu API trả về success thì lưu danh sách CV vào cvList:

```
setCVList(data.cvs);
```

Hàm handleChangeStatus(id, status) dùng để đổi trạng thái CV. Nó gửi request PATCH tới:

```
/company/cv/change-status/${id}
```

Body gửi lên là:

```
{
  status: status
}
```

Nếu lỗi thì hiện thông báo lỗi bằng toast.error.

Nếu thành công thì hiện thông báo thành công và tăng countUpdate lên 1 để gọi lại API danh sách CV:

```
setCountUpdate(countUpdate + 1);
```

Tóm lại: **lấy danh sách CV → lưu vào state → đổi trạng thái CV → tải lại danh sách CV sau khi đổi thành công.**

2.1.2. Xử lý bên phía Backend

```
router.patch(
  "/cv/change-status/:id",
  authMiddleware.verifyTokenCompany,
  companyController.changeStatusCVPatch
)
```

Đây là **route backend** nhận request từ frontend:

```
router.patch(  
  "/cv/change-status/:id",  
  authMiddleware.verifyTokenCompany,  
  companyController.changeStatusCVPatch  
)
```

Ý nghĩa:

PATCH là phương thức dùng để **cập nhật một phần dữ liệu**.

/cv/change-status/:id là đường dẫn API, trong đó :id là id của CV.

authMiddleware.verifyTokenCompany kiểm tra công ty đã đăng nhập/chứng thực token chưa.

companyController.changeStatusCVPatch là hàm xử lý chính: lấy id, lấy status từ body, rồi cập nhật trạng thái CV.

Nó khớp với frontend này:

```
fetch(`${API_URL}/company/cv/change-status/${id}`, {  
  method: "PATCH",  
  body: JSON.stringify({ status })  
})
```

Tóm lại: **frontend gửi id CV + status mới lên route này, backend kiểm tra token công ty rồi cập nhật trạng thái CV.**

```

export const changeStatusCVPatch = async (req: AccountRequest, res: Response) => {
  try {
    const cvId = req.params.id;
    const companyId = req.account.id;
    const statusCV = req.body.status;

    const infoCV = await CV.findOne({
      _id: cvId
    })

    if(!infoCV) {
      res.json({
        code: "error",
        message: "Dữ liệu không hợp lệ!"
      })
      return;
    }

    const infoJob = await Job.findOne({
      _id: infoCV.jobId,
      companyId: companyId
    });

    if(!infoJob) {
      res.json({
        code: "error",
        message: "Dữ liệu không hợp lệ!"
      })
      return;
    }

    await CV.updateOne({
      _id: cvId
    }, {
      status: statusCV
    })

    res.json({
      code: "success",
      message: "Đã cập nhật trạng thái!"
    })
  } catch (error) {
    console.log(error);
    res.json({
      code: "error",
      message: "Dữ liệu không hợp lệ!"
    })
  }
}

```

Hàm này là phần **controller backend** xử lý việc đổi trạng thái CV.

1. Lấy dữ liệu từ request

```

const cvId = req.params.id;
const companyId = req.account.id;
const statusCV = req.body.status;

```

Ví dụ frontend gọi:

PATCH /cv/change-status/123

Body:

```

{
  "status": "approved"
}

```

thì:

```
cvId = "123"  
statusCV = "approved"  
companyId = id công ty đăng nhập
```

2. Kiểm tra CV có tồn tại không

```
const infoCV = await CV.findOne({  
  _id: cvId  
})
```

Tìm CV theo id.

Nếu không tìm thấy:

```
if(!infoCV){  
  res.json({  
    code: "error",  
    message: "Dữ liệu không hợp lệ!"  
  })  
  return;  
}
```

=> Trả lỗi và dừng hàm.

3. Kiểm tra CV có thuộc công ty này không

Mỗi CV ứng tuyển vào một Job.

Lấy Job của CV:

```
const infoJob = await Job.findOne({  
  _id: infoCV.jobId,  
  companyId: companyId  
});
```

Điều kiện:

- Job phải tồn tại
- Job phải thuộc công ty đang đăng nhập

Nếu không:

```

if(!infoJob){
  res.json({
    code: "error",
    message: "Dữ liệu không hợp lệ!"
  })
  return;
}

```

=> Công ty không được sửa CV của công ty khác.

4. Cập nhật trạng thái CV

```

await CV.updateOne(
  {
    _id: cvId
  },
  {
    status: statusCV
  }
)

```

Ví dụ:

statusCV = "approved"

thì MongoDB sẽ cập nhật:

```

{
  "_id": "123",
  "status": "approved"
}

```

5. Trả kết quả thành công

```

res.json({
  code: "success",
  message: "Đã cập nhật trạng thái"
})

```

Frontend nhận được:

```

{
  "code": "success",
  "message": "Đã cập nhật trạng thái"
}

```


Sau đó frontend:

```
toast.success(data.message);
setCountUpdate(countUpdate + 1);
```

=> hiện thông báo và tải lại danh sách CV.

Luồng tổng thể

Frontend

```
|
| PATCH /cv/change-status/:id
| status = approved
v
```

Controller

```
|
|-- Lấy cvId
|-- Lấy companyId từ token
|-- Tìm CV
|-- Tìm Job của CV
|-- Kiểm tra Job thuộc company hiện tại
|-- Update status
v
```

MongoDB

```
|
v

{
  code: "success",
  message: "Đã cập nhật trạng thái"
}
```

Mục đích của đoạn:

```
const infoJob = await Job.findOne({
  _id: infoCV.jobId,
  companyId: companyId
});
```

là **bảo mật**: chỉ cho phép công ty sở hữu Job đó được đổi trạng thái CV ứng tuyển vào Job đó.

2.1.3. Kết quả

Quản lý CV

Công việc 4
Ứng viên: **Le Van A**
✉ levana@gmail.com
☎ 0954656564
600\$ - 900\$
👤 Fresher
🏢 Tại văn phòng
👁 Đã xem
🕒 Chưa duyệt

Xem Duyệt

Từ chối Xóa

Quản lý CV

Công việc 4
Ứng viên: **Le Van A**
✉ levana@gmail.com
☎ 0954656564
600\$ - 900\$
👤 Fresher
🏢 Tại văn phòng
👁 Đã xem
✅ Đã duyệt

Xem Từ chối

Xóa

Quản lý CV

Công việc 4
Ứng viên: **Le Van A**
✉ levana@gmail.com
☎ 0954656564
600\$ - 900\$
👤 Fresher
🏢 Tại văn phòng
👁 Đã xem
🚫 Từ chối

Xem Duyệt Xóa

Người quản trị có thể thao tác duyệt hoặc từ chối CV mà mình muốn

2.2. Tính năng xóa (Trang quản lý CV)

2.2.1. Xử lý bên phía Frontend

```
const handleDelete = (id: string) => {  
  fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/cv/delete/${id}`, {  
    method: "DELETE",  
    credentials: "include", // Gửi kèm cookie  
  })  
  .then(res => res.json())  
  .then(data => {  
    if(data.code == "error") {  
      toast.error(data.message);  
    }  
  
    if(data.code == "success") {  
      toast.success(data.message);  
      setCountUpdate(countUpdate + 1);  
    }  
  })  
}
```

Đây là hàm frontend dùng để **xóa CV**.

```
const handleDelete = (id: string) => {
```

Nhận vào id của CV cần xóa.

```
fetch(`${API_URL}/company/cv/delete/${id}`, {  
  method: "DELETE",  
  credentials: "include"  
})
```

Gửi request DELETE lên backend, kèm cookie đăng nhập.

```
.then(res => res.json())  
.then(data => { ... })
```

Chuyển response về JSON rồi xử lý.

Nếu lỗi:

```
toast.error(data.message);
```

Nếu thành công:

```
toast.success(data.message);  
setCountUpdate(countUpdate + 1);
```

Hiện thông báo thành công và tăng countUpdate để useEffect chạy lại, từ đó tải lại danh sách CV.

Tóm lại: **bấm xóa CV → gọi API DELETE → backend xóa CV → frontend hiện thông báo → tải lại danh sách CV.**

2.2.2. Xử lý bên phía Backend

```
router.delete(  
  "/cv/delete/:id",  
  authMiddleware.verifyTokenCompany,  
  companyController.deleteCVDel  
)
```

Đây là **route backend** để xóa CV:

```
router.delete(  
  "/cv/delete/:id",  
  authMiddleware.verifyTokenCompany,  
  companyController.deleteCVDel  
)
```

Ý nghĩa:

DELETE là phương thức dùng để xóa dữ liệu.

/cv/delete/:id là đường dẫn API, trong đó :id là id của CV cần xóa.

authMiddleware.verifyTokenCompany kiểm tra công ty đã đăng nhập chưa.

companyController.deleteCVDel là hàm xử lý chính việc xóa CV.

Nó khớp với frontend:

```
fetch(`${API_URL}/company/cv/delete/${id}`, {  
  method: "DELETE",  
  credentials: "include"  
})
```

Tóm lại: **frontend gửi id CV cần xóa → backend kiểm tra token công ty → gọi controller để xóa CV.**

```

export const deleteCVDel = async (req: AccountRequest, res: Response) => {
  try {
    const cvId = req.params.id;
    const companyId = req.account.id;

    const infoCV = await CV.findOne({
      _id: cvId
    })

    if(!infoCV) {
      res.json({
        code: "error",
        message: "Dữ liệu không hợp lệ!"
      })
      return;
    }

    const infoJob = await Job.findOne({
      _id: infoCV.jobId,
      companyId: companyId
    });

    if(!infoJob) {
      res.json({
        code: "error",
        message: "Dữ liệu không hợp lệ!"
      })
      return;
    }

    await CV.deleteOne({
      _id: cvId
    })

    res.json({
      code: "success",
      message: "Đã xóa CV!"
    })
  } catch (error) {
    console.log(error);
    res.json({
      code: "error",
      message: "Dữ liệu không hợp lệ!"
    })
  }
}

```

Hàm này là **controller backend để xóa CV**.

Luồng xử lý:

const cvId = req.params.id;

const companyId = req.account.id;

Lấy id CV từ URL và lấy id công ty từ token đăng nhập.

const infoCV = await CV.findOne({ _id: cvId });

Kiểm tra CV có tồn tại không. Nếu không có thì trả lỗi.

```
const infoJob = await Job.findOne({
  _id: infoCV.jobId,
  companyId: companyId
});
```

Kiểm tra CV này thuộc Job của công ty đang đăng nhập không.

Đoạn này để tránh công ty A xóa CV của công ty B.

```
await CV.deleteOne({ _id: cvId });
```

Nếu hợp lệ thì xóa CV khỏi database.

```
res.json({
  code: "success",
  message: "Đã xóa CV!"
});
```

Trả kết quả thành công về frontend.

Tóm lại: lấy id CV → kiểm tra CV tồn tại → kiểm tra CV thuộc công ty hiện tại → xóa CV → trả thông báo thành công.

2.2.3. Kết quả



Quản lý CV

Công việc 4
Ứng viên: **Le Van A**
✉ levana@gmail.com
☎ 0954656564
600\$ - 900\$
👤 Fresher
📁 Tại văn phòng
👁 Đã xem
✅ **Từ chối**

XemDuyệtXóa

Khi ấn xóa CV tương ứng sẽ mất đi và thông báo Đã xóa CV! Sẽ hiện ra

2.3. Quản lý CV đã gửi.

2.3.1. Xử lý bên phía Frontend

```
export const CVList = () => {  
  const [cvList, setCVList] = useState<any[]>([]);  
  
  useEffect(() => {  
    fetch(`${process.env.NEXT_PUBLIC_API_URL}/user/cv/list`, {  
      method: "GET",  
      credentials: "include", // Gửi kèm cookie  
    })  
      .then(res => res.json())  
      .then(data => {  
        if(data.code == "success") {  
          setCVList(data.cvs);  
        }  
      })  
    }, []);  
}
```

```

return (
  <div className="grid lg:grid-cols-3 sm:grid-cols-2 grid-cols-1 gap-[20px]">
    {cvList.map(item) => {
      const position = positionList.find(pos => pos.value == item.jobPosition);
      const workingForm = workingFormList.find(work => work.value == item.jobWorkingForm);
      const status = cvStatusList.find(st => st.value == item.status);

      return (
        <div
          key={item.id}
          className="border border-[#DEDEDE] rounded-[8px] flex flex-col relative truncate"
          style={{
            background: "linear-gradient(180deg, #F6F6F6 2.38%, #FFFFFF 70.43%)"
          }}
        >
          
          <h3 className="mt-[20px] mx-[16px] font-[700] text-[18px] text-[#121212] text-center flex-1 whitespace-normal line-clamp-2">
            {item.jobTitle}
          </h3>
          <div className="mt-[12px] text-center font-[400] text-[14px] text-black">
            Công ty: <span className="font-[700]">{item.companyName}</span>
          </div>
          <div className="mt-[6px] text-center font-[600] text-[16px] text-[#0088FF]">
            {item.jobSalaryMin.toLocaleString("vi-VN")}$ - {item.jobSalaryMax.toLocaleString("vi-VN")}$
          </div>
          <div className="mt-[6px] flex justify-center items-center gap-[8px] font-[400] text-[14px] text-[#121212]">
            <FaUserTie className="text-[16px]" /> {position?.label}
          </div>
          <div className="mt-[6px] flex justify-center items-center gap-[8px] font-[400] text-[14px] text-[#121212]">
            <FaBriefcase className="text-[16px]" /> {workingForm?.label}
          </div>
          <div
            className="mt-[6px] flex justify-center items-center gap-[8px] font-[400] text-[14px]"
            style={{
              color: status?.color
            }}
          >
            <FaCircleCheck className="text-[16px]" /> {status?.label}
          </div>
          <div className="flex flex-wrap items-center justify-center gap-[8px] mt-[12px] mb-[20px] mx-[10px]">
            <Link href="/user-manage/cv/detail/${item.id}" className="bg-[#0088FF] rounded-[4px] font-[400] text-[14px] text-white inline-block py-[8px] px-[20px]">
              Xem
            </Link>
            <Link href="#" className="bg-[#FF8080] rounded-[4px] font-[400] text-[14px] text-white inline-block py-[8px] px-[20px]">
              Xóa
            </Link>
          </div>
        </div>
      )
    }}
  </div>
)

```

```

<div className="mt-[30px]">
  <select name="" className="border border-[#DEDEDE] rounded-[8px] py-[12px] px-[18px] font-[400] text-[16px] text-[#414042]">
    <option value="">Trang 1</option>
    <option value="">Trang 2</option>
    <option value="">Trang 3</option>
  </select>
</div>
</>

```

Code này là trang **user xem danh sách CV đã ứng tuyển**.

Phần đầu:

```
const [cvList, setCVList] = useState<any[]>([]);
```

Tạo state lưu danh sách CV.

```
useEffect(() => {
  fetch(`${API_URL}/user/cv/list`, {
    method: "GET",
    credentials: "include"
  })
})
```

Khi component chạy lần đầu, gọi API lấy danh sách CV của user.

Nếu thành công:


```
setCVList(data.cvs);
```

lưu CV vào cvList.

Phần giao diện:

```
{cvList.map((item) => { ... })}
```

Duyệt từng CV và hiển thị thành card.

Trong mỗi card có:

position

workingForm

status

3 biến này dùng để tìm label tương ứng từ các mảng cấu hình như positionList, workingFormList, statusList.

Hiển thị thông tin:

```
{item.jobTitle}
```

```
{item.companyName}
```

```
{item.jobSalaryMin} - {item.jobSalaryMax}
```

```
{position?.label}
```

```
{workingForm?.label}
```

```
{status?.label}
```

Hai nút:

Xem

Xóa

Xem dẫn tới trang chi tiết CV:

```
/user-manage/cv/detail/${item.id}
```

Xóa hiện tại mới là link #, chưa gọi hàm xóa.

Cuối trang có phân trang:

```
<select>
```

```
<option>Trang 1</option>
```

```
<option>Trang 2</option>
```

```
<option>Trang 3</option>
```

```
</select>
```

Tóm lại: gọi API lấy CV của user → lưu vào state → map ra các card CV → hiển thị thông tin + trạng thái + nút xem/xóa.

2.3.2. Xử lý bên phía backend

```
router.get(  
  "/cv/list",  
  authMiddleware.verifyTokenUser,  
  userController.listCV  
)
```

Đây là route backend để lấy danh sách CV của user.

```
router.get(  
  "/cv/list",  
  authMiddleware.verifyTokenUser,  
  userController.listCV  
)
```

1. Route

"/cv/list"

Khi frontend gọi:

```
fetch(`${API_URL}/user/cv/list`)
```

thì request sẽ đi vào route này.

2. Middleware xác thực

authMiddleware.verifyTokenUser

Kiểm tra user đã đăng nhập chưa.

Nếu token hợp lệ:

```
req.account = {  
  id: "..."  
}
```

được gắn vào request.

Nếu chưa đăng nhập:

```
{  
  "code": "error",  
  "message": "Unauthorized"  
}
```

3. Controller

UserController.listCV

Hàm này sẽ:

- Lấy userId từ token
- Tìm các CV của user trong database
- Lấy thêm thông tin Job nếu cần
- Trả danh sách CV về frontend

Thông thường luồng sẽ giống:

```
const userId = req.account.id;
```

```
const cvs = await CV.find({  
  userId: userId  
});
```

```
res.json({  
  code: "success",  
  cvs: cvs  
});
```

Luồng tổng thể

Frontend

```
|  
| GET /user/cv/list  
v
```

verifyTokenUser

```
|  
| Kiểm tra token  
| Lấy userId  
v
```

listCV

```
|  
| Query database  
| Lấy CV của user  
v
```

```
{  
  code: "success",
```

```
    cvs: [...]  
  }
```

Sau đó frontend:

```
.then(data => {  
  if(data.code == "success") {  
    setCVList(data.cvs);  
  }  
})
```

=> lưu danh sách CV vào cvList và render ra các card trên giao diện.

```

export const listCV = async (req: AccountRequest, res: Response) => {
  try {
    const email = req.account.email;

    const cvs = await CV
      .find({
        email: email
      })
      .sort({
        createdAt: "desc"
      });

    const dataFinal = [];

    for (const item of cvs) {
      const jobDetail = await Job.findOne({
        _id: item.jobId
      })

      const companyDetail = await AccountCompany.findOne({
        _id: jobDetail?.companyId
      })

      if(jobDetail && companyDetail) {
        const itemFinal = {
          id: item.id,
          jobTitle: jobDetail.title,
          companyName: companyDetail.companyName,
          jobSalaryMin: jobDetail.salaryMin,
          jobSalaryMax: jobDetail.salaryMax,
          jobPosition: jobDetail.position,
          jobWorkingForm: jobDetail.workingForm,
          status: item.status,
        };

        dataFinal.push(itemFinal);
      }
    }

    res.json({
      code: "success",
      message: "Thành công!",
      cvs: dataFinal
    })
  } catch (error) {
    console.log(error);
    res.json({
      code: "error",
      message: "Dữ liệu không hợp lệ!"
    })
  }
}

```

Hàm listCV dùng để lấy danh sách CV mà user đã ứng tuyển.

Luồng chính:

const email = req.account.email;

Lấy email user từ token đăng nhập.

```
const cvs = await CV.find({ email: email }).sort({ createdAt: "desc" });
```

Tìm tất cả CV có email của user đó, sắp xếp CV mới nhất lên trước.

```
for (const item of cvs) {
```

Duyệt từng CV.

```
const jobDetail = await Job.findOne({ _id: item.jobId });
```

Lấy thông tin công việc mà CV đã ứng tuyển.

```
const companyDetail = await AccountCompany.findOne({  
  _id: jobDetail?.companyId  
});
```

Lấy thông tin công ty đăng bài tuyển dụng đó.

Nếu có đủ jobDetail và companyDetail, tạo object mới:

```
const itemFinal = {  
  id: item.id,  
  jobTitle: jobDetail.title,  
  companyName: companyDetail.companyName,  
  jobSalaryMin: jobDetail.salaryMin,  
  jobSalaryMax: jobDetail.salaryMax,  
  jobPosition: jobDetail.position,  
  jobWorkingForm: jobDetail.workingForm,  
  status: item.status,  
};
```

Rồi thêm vào mảng kết quả:

```
dataFinal.push(itemFinal);
```

Cuối cùng trả về frontend:

```
res.json({  
  code: "success",  
  message: "Thành công!",  
  cvs: dataFinal  
});
```

Tóm lại: lấy email user → tìm CV của user → lấy thêm thông tin Job và Công ty → gom dữ liệu lại → trả danh sách CV cho frontend hiển thị.

2.3.3. Kết quả



Danh sách đã gửi sẽ hiện ra để quản lý

3. Kết quả đạt được

Sau khi thực hiện, các chức năng quản lý CV trong tuần đã hoàn thành được các yêu cầu cơ bản.

Về phía nhà tuyển dụng, hệ thống đã hỗ trợ thao tác duyệt và từ chối CV. Khi nhà tuyển dụng chọn trạng thái mới cho một CV, Frontend gửi request PATCH lên Backend kèm theo id CV và trạng thái cần cập nhật. Backend kiểm tra token công ty, kiểm tra CV có tồn tại hay không, đồng thời kiểm tra CV đó có thuộc công việc của công ty đang đăng nhập không. Nếu hợp lệ, hệ thống cập nhật trạng thái CV trong cơ sở dữ liệu và trả thông báo thành công về Frontend.

Ngoài ra, chức năng xóa CV cũng đã được xây dựng. Khi nhà tuyển dụng bấm xóa, Frontend gửi request DELETE lên Backend. Backend tiếp tục kiểm tra quyền sở hữu thông qua công việc liên quan đến CV. Nếu CV thuộc đúng công ty đang đăng nhập, hệ thống sẽ xóa CV khỏi cơ sở dữ liệu và cập nhật lại danh sách CV trên giao diện.

Về phía ứng viên, hệ thống đã xây dựng được trang quản lý CV đã gửi. Trang này cho phép ứng viên xem lại các CV mà mình đã ứng tuyển. Dữ liệu hiển thị bao gồm tên công việc, tên công ty, mức lương, vị trí công việc, hình thức làm việc và trạng thái xử lý CV. Danh sách CV được lấy từ Backend dựa trên email của tài khoản user đang đăng nhập và được sắp xếp theo thời gian mới nhất.

Kết quả cuối cùng là hệ thống đã hoàn thiện thêm các chức năng quan trọng trong luồng quản lý CV, bao gồm: nhà tuyển dụng có thể duyệt, từ chối, xóa CV; ứng viên có thể xem lại danh sách CV đã gửi và theo dõi trạng thái xử lý.

4. Khó khăn, vướng mắc

Trong quá trình thực hiện các chức năng tuần này, em gặp một số khó khăn như sau:

Thứ nhất, khi xử lý chức năng duyệt và từ chối CV, cần đồng bộ đúng trạng thái giữa Frontend và Backend. Nếu trạng thái gửi lên không đúng với dữ liệu mà hệ thống quy định thì việc hiển thị hoặc cập nhật trạng thái có thể bị sai.

Thứ hai, sau khi cập nhật trạng thái hoặc xóa CV, giao diện cần được tải lại để hiển thị dữ liệu mới nhất. Nếu không xử lý cập nhật lại danh sách, người dùng có thể vẫn nhìn thấy dữ liệu cũ trên giao diện.

Thứ ba, ở phía Backend cần kiểm tra quyền thao tác dữ liệu. Không chỉ cần tìm CV theo id, mà còn phải kiểm tra công việc liên quan đến CV đó có thuộc về công ty đang đăng nhập hay không. Nếu bỏ qua bước này thì một công ty có thể sửa hoặc xóa CV của công ty khác.

Thứ tư, khi xây dựng trang quản lý CV đã gửi cho ứng viên, dữ liệu hiển thị không chỉ nằm trong bảng CV mà còn cần lấy thêm thông tin từ bảng Job và AccountCompany. Vì vậy cần xử lý kết hợp dữ liệu từ nhiều collection khác nhau.

Thứ năm, trong một số trường hợp dữ liệu CV, Job hoặc Company không tồn tại, chương trình cần kiểm tra cẩn thận để tránh lỗi khi truy cập vào thuộc tính của giá trị null hoặc undefined.

5. Hướng giải quyết

Để xử lý các khó khăn trên, em đã áp dụng một số giải pháp sau:

Ở phía Frontend, em sử dụng state countUpdate để kiểm soát việc tải lại dữ liệu. Sau khi cập nhật trạng thái CV hoặc xóa CV thành công, giá trị countUpdate được tăng lên, từ đó làm cho useEffect chạy lại và gọi API lấy danh sách CV mới nhất.

Đối với chức năng duyệt và từ chối CV, em gửi request PATCH lên API `/company/cv/change-status/:id`, trong đó id là id của CV cần cập nhật, còn trạng thái mới được gửi trong phần body. Khi Backend trả về kết quả thành công, hệ thống hiển thị thông báo bằng toast.success.

Đối với chức năng xóa CV, em sử dụng phương thức DELETE với API `/company/cv/delete/:id`. Sau khi xóa thành công, Frontend cũng hiển thị thông báo và tải lại danh sách CV để giao diện được cập nhật.

Ở phía Backend, em sử dụng middleware `authMiddleware.verifyTokenCompany` để kiểm tra tài khoản công ty trước khi cho phép thực hiện các thao tác cập nhật trạng thái hoặc xóa CV. Trong controller, em kiểm tra CV có tồn tại không, sau đó tiếp tục tìm Job tương ứng với điều kiện `companyId` phải trùng với công ty đang đăng nhập. Cách xử lý này giúp đảm bảo chỉ công ty sở hữu công việc đó mới được thao tác với CV.

Đối với trang quản lý CV đã gửi của ứng viên, em sử dụng middleware `authMiddleware.verifyTokenUser` để xác thực tài khoản user. Backend lấy email từ token, tìm danh sách CV theo email đó, sau đó lấy thêm thông tin công việc và công ty để trả về dữ liệu đầy đủ cho Frontend hiển thị.

Ngoài ra, em sử dụng `try...catch` trong các controller để bắt lỗi phát sinh trong quá trình truy vấn database. Khi có lỗi hoặc dữ liệu không hợp lệ, API sẽ trả về thông báo lỗi thay vì làm chương trình bị dừng đột ngột.

6. Kế hoạch tuần tiếp theo

Hiện tại các website tuyển dụng việc làm cũng đã gần như hoàn thành, trong tuần tới em sẽ bắt đầu hoàn thành báo cáo cuối kỳ cho bài tập lớn của mình !